Annex of the paper:
Benefits and Drawbacks of Reference Architectures

Silverio Martínez-Fernández[1], Claudia Ayala[1], Xavier Franch[1], Helena Marques[2]

[1] GESSI Research Group, Universitat Politècnica de Catalunya, Barcelona, Spain
{smartinez,cayala,franch}@essi.upc.edu

[2] *everis*, Barcelona, Spain
hmartinm@everis.com

1 Industrial context

This study has its origin in an ongoing action-research initiative among our research group and *everis*. As a consulting company, *everis* offers solutions for big businesses (e.g., banks, insurance companies, public administration, utilities, and industrial organizations) that provide a wide spectrum of services to their clients. Given the complexity of the resulting IT systems, that integrate bespoke applications with commercial packages, these companies need high-quality software architectures, and this is the service that they hire to *everis*. The solution that *everis* provides is based on the deployment of an RA in the company, from which concrete software architectures are derived and used in a wide spectrum of applications.

In this context, *everis* commissioned our research group to systematically gather empirical evidence about the benefits and drawbacks of the adoption of RAs for their clients, in order to avoid just relying on anecdotal evidences.

To gather such empirical evidence, it is necessary to contact RA stakeholders. *everis* distinguishes three essential roles: **software architects** that cooperatively work to figure out an RA based on the *everis* reference model to accomplish the desired quality attributes and architecturally-significant requirements of the client organization; **architecture developers** that are responsible for coding, maintaining, integrating, testing and documenting the RA software components; and **application builders** that take the RA reusable components and instantiate them to build concrete software architectures for systems. Fig. 1 shows these stakeholders. It shows how software architects and architecture developers form the **RA team**, whilst application builders form **concrete software architecture teams** (one for every system). Both teams may combine professionals from *everis* (i.e., development organization [4]) and from the client organization (i.e., acquisition organization [4]).

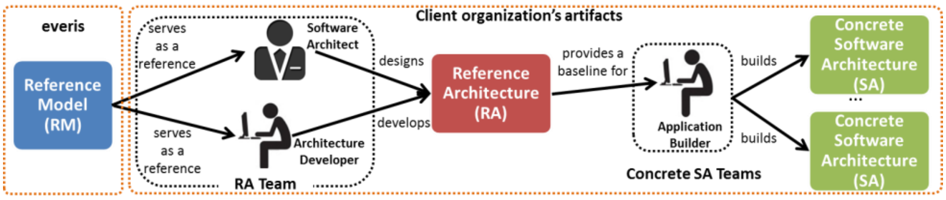

Fig. 1. Essential stakeholders for software RA analysis.

2 Research Design

Objectives of the study. The overall objective of this study is to provide evidence from different stakeholders about potential benefits and drawbacks of adopting software RAs in order to understand the industrial perception of the benefits discussed in the literature of RAs. This general objective was broken down into more specific research questions (see Table 1) focused on RA benefits and drawbacks.

Table 1. Research Questions of our study.

RQ1	What are the main benefits of RAs for the development and maintenance of software applications as seen by different stakeholders?
RQ2	What are the main drawbacks of RAs for the development and maintenance of software applications as seen by different stakeholders?

Taking into account that the nature of our research questions was mainly descriptive (i.e., they were aimed to identify and describe the perceptions of different stakeholders) we decided to carry out a qualitative research approach using different methods to collect the data.

Sampling. The target population was: practitioners that played an important role on the RA design, its use for creating concrete software architectures, or the associated development tasks. Therefore, with the help of *everis* we identified 3 different target roles: software architects, architecture developers and application builders. In addition, it was important to focus our questions on a specific project, so we could better understand the context of the perceptions of each role. Thus, a manager from *everis* selected 9 projects from different client organizations on the basis of its suitability and feasibility to contact at least with one person playing each of the targeted roles (only in one project we could not contact application builders). The manager from *everis* provided us with detailed information of the projects and then contacted with potential participants for agreeing on their participation. We finally ended up with 28 people that participated in the selected projects and played any of the stated roles. Table 2 gives information about the projects of client organizations.

Data Collection and Instruments. Semi-structured interviews were used for software architects because they have higher knowledge about the architectural aspects, and it was crucial to understand the rationale behind their strategic decisions. Semi-structured interviews provided us the opportunity to carefully prepare an interview guide and to add follow-up questions whenever we needed clarification or further explanations. Prior to the interview, we sent by email to each participant a kindly reminder of our meeting and requested them their personal information and documentation of the RA project. This was useful to prepare the interview and save time.

On the other hand, it was impossible to contact personally architecture developers and application builders. Hence, online questionnaires were used for collecting data from them. It could be done because most of their responses can be previously listed, what simplified the data collection process. In any event, we provided room to add any comment or observation to all the listed questions from the questionnaire.

Table 2. Overview of the projects executed in client organizations

Id org.	Participants			Main domain	RA Project Context
	SA	AD	AB		
A	1	1	1	Industry	To develop an RA for web-based applications to allow vendors to retrieve/insert information about clients in a department store.
B	1	1	1	Banking	To define, implement and maintain an RA that allows the construction of multi-platform applications that are fast, satisfy practices of the market and support transaction processing in a bank.
C	1	1	1	Banking	To define, design and implement a new RA for developing and implementing applications in a bank.
D	1	1	1	Insurance	To evolve the RA to support a wide scope of functional and geographical system requirements for internal request for proposals.
E	1	1	1	Public sector	An RA for developing Java web applications, with flexible front-end, integration between systems and batch processes
F	1	1	2	Public sector	To offer and maintain an RA that serves as a common framework for the development of systems in a public administration.
G	1	1	0	Public sector	To define a robust, flexible and standard RA that enhances reusability, reducing development costs of each new system.
H	1	1	2	Insurance	An RA that allows building efficient, agile, robust and extensible systems in an insurance company.
I	1	1	1	Public sector	An RA that helps to meet strategic goals and optimization and orchestration of business processes in a utility organization.

[1] Legend: Software Architect (SA), RA Developer (AD), Application Builder (AB).

Data analysis. To perform data analysis, the research team held several discussion meetings during and after data collection. We incrementally processed the manual transcriptions of all interviews and automatically get the data from the online questionnaires. We based our analysis on grounded theory techniques as constant comparison and cross case analysis [7]. These techniques are well-fitting in situations where the researcher does not want to use pre-conceived ideas, and instead is driven by the desire to capture all facets of the collected data and to allow the propositions to emerge from the data. Fig. 1 shows the context of our study, two different cases (i.e., what is studied?) and the units of analysis partitioned in three groups based on their role (i.e., software architects, architecture developers or application builders).

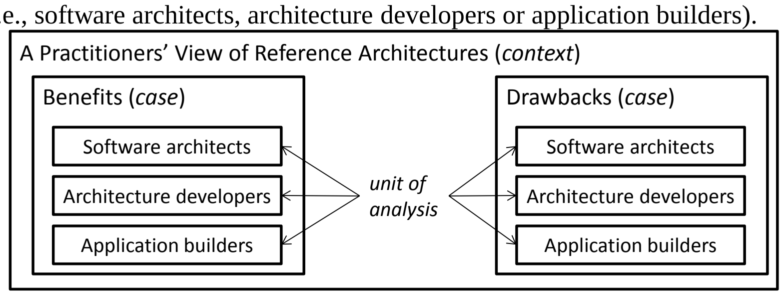

Fig. 2. Data analysis protocol with embedded units of analysis.

The data analysis consisted of two steps. First, analysis was driven by coding pieces of data as the constant comparison method requires [7]. We first read the interview transcripts and the data from the questionnaires and attached a coding word to a portion of the text – a phrase or a paragraph. The codes were selected to reflect the meaning of the respective portion of the interview text to a specific research question. Second, we performed cross-case analysis [7] to see the different views from multiple stakeholders over the same case (i.e., benefits/drawbacks). We clustered all pieces of text that relate to the same code to analyze it in a consistent and systematic way.

Limitations of the study. This section discusses possible threats to validity in terms of construct, internal and external validity as well as reliability, as proposed by Yin [10] and Runeson and Höst [6]. It also emphasizes the mitigation actions used.

Construct validity. To strengthen this aspect we made sure of performing a rigorous planning of the study and establishing a solid protocol for data collection and data analysis by following guidelines for software engineering [2][6][7][9]. Our protocol included specific mitigation actions for evaluation apprehension by ensuring the confidentiality of the answers. In addition, the instruments used to gather data (i.e., the interview guide and the questionnaires) were piloted and enhanced to ensure their effectiveness. For instance, we studied what *everis*' stakeholders understand by RAs in order to design the instruments.

Internal validity. One of the main relevant decisions directly affected the sampling approach as we decided to first choose *everis*' projects and then participants that covered the roles we were looking for. In this way, we ensured that each participant would focus his/her answers on the context of the defined project. This would allow a better interpretation and assessment of contextual information. It would otherwise have been very difficult to interpret certain decisions or influential factors related to the nature of the projects. We are aware that some possible biases may be related to this strategy, for instance that the *everis*' manager chose the most successful projects as sampling. To minimize this, we explained her/him the importance of having a representative sampling of the projects they perform in order to obtain reliable data.

Furthermore, regarding individuals that participated in the study, there is always the possibility that they forget something or do not explicitly state it when he/she is asked about it. To reduce this risk: 1) in the case of the interviews, we discussed some potential topics that might be omitted by the respondents, and paid particular attention to them during the interviews in order to ask for clarifications if necessary; 2) in the case of the online questionnaires, we designed them in such a way that the respondent must answer all the corresponding questions while he/she could complete the questionnaire at any time, so it gives them the possibility of consulting registries and documentation in case he/she needs to remember something; 3) in all cases we performed triangulation by adding questions aimed to confirm the correctness of the answers.

We put forward several mitigation strategies. First, recording and transcribing all interviews contributed to a better understanding and assessment of the data gathered. Second, to reduce the potential researcher bias, several meetings were held among the researchers in order to discuss the results.

External validity. First, it is important to stand out that our results do not attempt to be generalized outside *everis*' context; however we think that it could be possible to observe similar experiences in projects and companies with similar contexts (i.e., organizations where the multiplicity of software systems -systems developed at multiple locations, by multiple vendors and across multiple organizations- triggers a need for life-cycle support for all systems by means of RAs as indicated in [3][5]). As Seddon suggests: "if the forces within an organization that drove observed behavior are likely to exist in other organizations, it is likely that those other organizations, too, will exhibit similar behavior" [8]. Moreover, we acknowledge that several other factors may have influenced the roles perceptions (as organizational processes and policies, cultural issues, etc.). We, therefore, think that more research is needed to understand the relationship between the roles perception and the organizations.

Reliability. In order to strengthen this aspect we addressed the validity of the study. Besides the strategies above, we maintained a detailed protocol, conducted the survey tasks (data collection and analysis) by at least two of the authors, spent sufficient time with the study, and gave sufficient concern to the analysis of all responses.

References

1. Angelov, S., Grefen, P., Greefhorst, D.: A framework for analysis and design of software reference architectures. Information and Software Technology 54(4), 417 - 431 (2012).
2. Ciolkowski, M., Laitenberger, O., Vegas, S., Biffl, S.: Practical experiences in the design and conduct of surveys in empirical software engineering. ESERNET, pp. 104-128 (2003)
3. Cloutier, R., Muller, G., Verma, D., Nilchiani, R., Hole, E., Bone, M.: The concept of reference architectures. Systems Engineering 13(1), 14-27 (2010)
4. Gallagher, B.: Using the architecture tradeoff analysis methodsm to evaluate a reference architecture: A case study. SEI CMU Tech. rep., DTIC Document (2000)
5. Muller, G., Laar, P.: Researching reference architectures. CSER (2009)
6. Runeson, P., Höst, M.: Guidelines for conducting and reporting case study research in software engineering. Empirical Software Engineering 14(2), 131-164 (2009)
7. Seaman, C.B.: Qualitative methods in empirical studies of software engineering. IEEE Transactions on Software Engineering 25(4), 557-572 (1999)
8. Seddon, P., Scheepers, P.: Towards the improved treatment of generalization of knowledge claims in IS research: drawing general conclusions from samples, EJIS, pp. 1-16, (2011)
9. Wohlin, C., Höst, M., Henningsson, K.: Empirical research methods in software engineering. ESERNET, pp. 7-23 (2003)
10. Yin, R.K.: Case Study Research: Design and Methods. SAGE Publications (2009)